Proposed Redis Distributed Lock Implementation for Cache Refresh and Backend Calls

Subject: Proposed Redis Distributed Lock Implementation for Cache Refresh and Backend Calls

Hi Team,

As discussed, we have finalized the approach for implementing Redis Distributed Locking to prevent multiple concurrent requests from triggering duplicate backend calls when the cache is empty or requires a refresh. The objective is to ensure that only one request executes the backend call at a time for a given cache key, while other concurrent requests reuse cached data whenever possible.

1. Proposed Execution Flow

Step 1: Check the primary cache

For a normal request (**refresh=0**), the application first checks Redis for the requested data. If the data is available, it is returned immediately without invoking the backend. If the cache entry is missing, expired, or a refresh is requested, the flow proceeds to the distributed lock mechanism.

Step 2: Acquire the distributed lock

The application attempts to acquire a Redis lock using an atomic **SET NX** operation with an expiry time and a unique lock-owner value. If acquired, the request becomes the lock owner. If another request already holds the lock, the current request enters the waiting flow instead of immediately invoking the backend.

Step 3: Recheck the cache after acquiring the lock

Once a request acquires the lock, it checks Redis again before calling the backend. For a normal request, if the data has already been populated, the cached response is returned and the lock is released. If the data is still missing, or the request explicitly requires a refresh, the application invokes the backend. This second cache check prevents unnecessary backend calls caused by concurrent cache misses.

Step 4: Invoke the backend and update the cache

The lock-owning request invokes the backend API. When the backend call succeeds, the response is written to Redis with the appropriate TTL and returned to the caller. The lock is released in the **finally** block using a safe ownership check, ensuring that a request only releases its own lock.

Step 5: Handle concurrent requests

Requests that fail to acquire the lock enter a bounded waiting loop. The application periodically checks whether the lock is still present. If the cache is populated while another request holds the lock, the waiting request can return the cached response when allowed by the refresh policy. If the lock is released and the cache is still missing, the request rechecks the cache and attempts to acquire the lock. If the cache remains unavailable until the configured deadline, the request terminates with a timeout error.

2. Handling the refresh=1 Parameter

- **refresh=0:** Return cached data when available. If the cache misses, use the distributed lock to coordinate backend execution.
- **refresh=1:** Bypass the initial cache read and request fresh data from the backend, subject to the distributed lock mechanism. On successful execution, update the cache with the latest response.

For concurrent refresh requests, the implementation must preserve the intended refresh semantics while preventing duplicate backend calls. Requests waiting for an active refresh should follow the agreed refresh policy rather than automatically treating any existing cache entry as fresh.

3. Key Benefits

- Prevents duplicate backend calls caused by simultaneous cache misses.
- Coordinates cache population across multiple application instances or pods.
- Reduces unnecessary backend traffic and improves resource utilization.

- Allows waiting requests to reuse cached data when available.
- Supports explicit cache refresh requests through the refresh parameter.
- Uses bounded waiting to prevent requests from waiting indefinitely.

4. Important Implementation Considerations

The implementation will include the following safeguards:

- Atomic lock acquisition using Redis SET NX with an expiry.
- A unique lock-owner value for each acquisition.
- Safe lock release using a Lua script that verifies lock ownership.
- A second cache check after acquiring the lock.
- A bounded waiting period with appropriate timeout handling.
- Appropriate cache TTL and error handling when backend execution fails.

We will validate the behavior under concurrent requests, multiple application instances, backend failures, and cache expiration scenarios before finalizing the implementation.

Regards,
[Your Name]